

UNIVERSITETET I OSLO

Logistisk regresjon

Pierre Lison
Norsk Regnesentral (NR)

IN1160 – Introduksjon til
maskinlæring (Vår 2026)

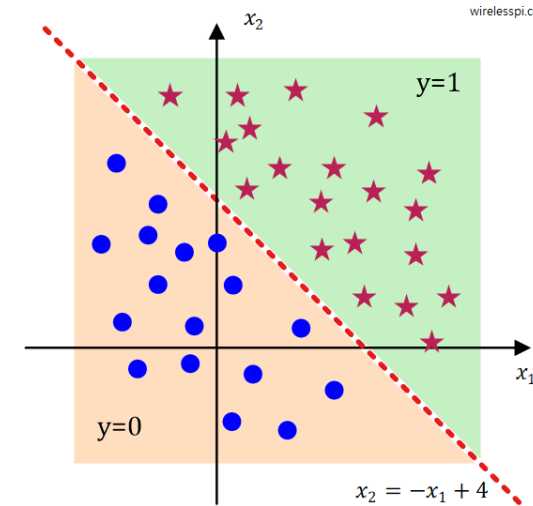
25.02.2026



I dag skal vi snakke om 2 temaer:

Lynkurs i matriseregning

Vektorer og matriser brukes overalt i maskinlæring – *det lønner seg å kunne grunnleggende matriseregning!*



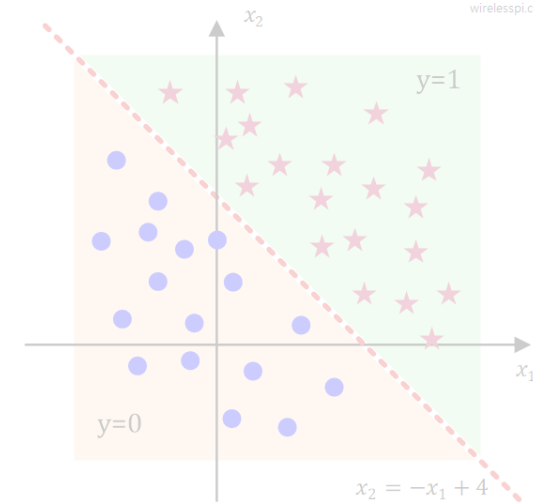
Logistisk regresjon

- En utvidelse av lineær regresjon som kan brukes til *klassifisering* i stedet for regresjon
- En viktig «brikke» i mange ML-modeller

I dag skal vi snakke om 2 temaer:

Lynkurs i matriseregning

Vektorer og matriser brukes
overalt i maskinlæring – *det
lønner seg å kunne
grunnleggende matriseregning!*



Logistisk regresjon

- En utvidelse av lineær regresjon som kan brukes til *klassifisering* i stedet for regresjon
- En viktig «brikke» i mange ML-modeller

Lynkurs i matriseregning

"Unfortunately, no one can be told what the Matrix is. You have to see it for yourself."

- Morpheus, "The Matrix"



Matematiske objekter

- **Skalar**: et enkelt tall, f.eks. -3.56
- **Vektor**: en rekke tall (av lengde m), f.eks. $\begin{bmatrix} -3.4 \\ 6.2 \\ \dots \\ 0.3 \end{bmatrix}$ Lengde m
- **Matrise**: en tabell (med m rad og n kolonner):

Antall kolonner n

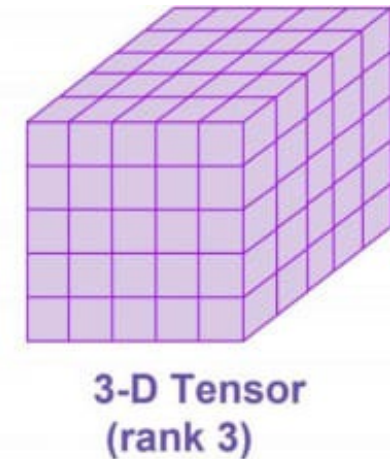
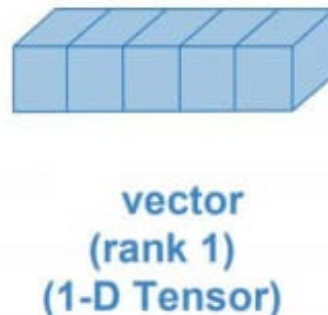
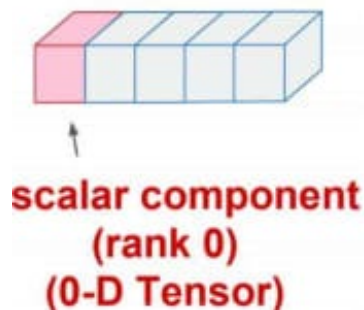
Antall rad m

$\begin{bmatrix} -1.4 & 4.5 & \dots & 2.3 \\ 8.4 & 0.2 & \dots & 0.5 \\ \vdots & \vdots & \ddots & \vdots \\ 2.8 & 1.3 & \dots & -4.9 \end{bmatrix}$	Celle (2,n)
--	-------------

Matematiske objekter

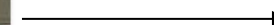
Disse kan generaliseres til **tensorer**:

- En skalar er en tensor med 0 dimensjon
- En vektor er en tensor med 1 dimensjon
- En matrise er en tensor med 2 dimensjoner
- Osv.



Matematiske objekter

- I maskinlæring bruker man tensorer (vektorer, matriser, osv.) for å representere både **data** og **parameterne**
- *Eksempel:* en video (uten lyd) = en 4-dimensjonal tensor :
 - Dimensjon 1 er tid (med "sampling" på en viss frekvens, f.eks. 50 frames/sek)
 - Dimensjon 2 og 3 er X- Y-akse i bildene
 - Dimensjon 4 er RGB verdiene



Blue					
Green					
	255	134	93	22	
Red	255	134	202	22	
	255	231	42	22	4
	123	94	83	2	92
	34	44	187	92	14
	34	76	232	124	14
	67	83	194	202	
					2
					30
					124
					142

Operasjoner

- Addisjon: $\begin{bmatrix} -3.4 \\ 6.2 \\ \dots \\ 0.3 \end{bmatrix} + \begin{bmatrix} -0.4 \\ -1.2 \\ \dots \\ 2.4 \end{bmatrix} = \begin{bmatrix} -3.8 \\ 5.0 \\ \dots \\ 2.7 \end{bmatrix}$
- Transposisjon : $\begin{bmatrix} -3.4 \\ 6.2 \\ \dots \\ 0.3 \end{bmatrix}^T = [-3.4 \quad 6.2 \quad \dots \quad 0.3]$
- Multiplikasjon med skalar: $2 * \begin{bmatrix} -3.4 \\ 6.2 \\ \dots \\ 0.3 \end{bmatrix} = \begin{bmatrix} -6.8 \\ 12.4 \\ \dots \\ 0.6 \end{bmatrix}$
- Elementvis produkt: $\begin{bmatrix} -2 \\ 1 \\ \dots \\ 0.3 \end{bmatrix} \circ \begin{bmatrix} -0.4 \\ -1.2 \\ \dots \\ 2.4 \end{bmatrix} = \begin{bmatrix} 0.8 \\ -1.2 \\ \dots \\ 0.72 \end{bmatrix}$ **Merk:** Elementvis produkt $\neq$ *matriseprodukt!* (mer om det snart)

Prikkprodukt

- Formelen: $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i$
- Regneprosedyre:
 - vi tar to vektorer (av samme lengde)

$$\mathbf{a} = \begin{bmatrix} -0.4 \\ -1.2 \\ 2.4 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} -3.4 \\ 6.2 \\ 0.3 \end{bmatrix}$$
$$\begin{array}{rcl} -0.4 * -3.4 & = & 1.36 \\ -1.2 * 6.2 & = & -7.44 \\ 2.4 * 0.3 & = & 0.72 \end{array}$$

- For hver posisjon i **ganger vi** a_i og b_i
- Og til slutt **legger vi sammen** alle tallene:

$$\mathbf{a} \cdot \mathbf{b} = (-0.4 * -3.4) + (-1.2 * 6.2) + (2.4 * 0.3) = -5.36$$

Matriseprodukt

- Prikkprodukt er mellom vektorer, men det finnes en lignende operasjon for matriser eller tensorer: *matriseproduktet*
- Matriseproduktet er definert mellom to matriser **A** og **B** hvor **A** har dimensjon $(m, \underline{n})$ og **B** er $(\underline{n}, p)$

Antall *kolonner* i **A** må være lik antall *rader* i **B**!

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

 n

$$\mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1p} \\ b_{21} & b_{22} & \cdots & b_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{np} \end{bmatrix}$$

 n

Matriseprodukt

Merk: dette lysarket bør helst sees i Powerpoint (og ikke PDF) pga. animasjonen som viser regneprosedyren

- **Hovedprinsipp:** Man tar en rad i **A** og en kolonne i **B** (altså to vektorer), og beregne *prikkproduktet* mellom de to
- Vi gjentar produktet for alle kolonnene $1, \dots, p$ i **B**
- Og for alle radene $1, \dots, m$ i **A**

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1p} \\ b_{21} & b_{22} & \cdots & b_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{np} \end{bmatrix} \Rightarrow \mathbf{AB} = \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1p} \\ c_{21} & c_{22} & \cdots & c_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m1} & c_{m2} & \cdots & c_{mp} \end{bmatrix}$$

hvor hver $c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$
(altså prikkproduktet mellom rad $a_{i.}$ og kolonne $b_{.j}$)

Mentimeter

Gitt disse to matrisene:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \text{ og } \mathbf{B} = \begin{bmatrix} 5 & -1 \\ 1 & 0 \\ -2 & 3 \end{bmatrix}$$

Spørsmål 1: Hva er dimensjon til $\mathbf{C} = \mathbf{AB}$?

Spørsmål 2: Regn ut $\mathbf{C}$

Matriseprodukt

Forrige uke så vi at vi kunne skrive n datapunkter $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}$ som en matrise med n rad og p kolonner (hvor p er antall trekk per input):

$$\mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_p^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_p^{(2)} \\ \dots & \dots & \dots & \dots \\ x_1^{(n)} & x_2^{(n)} & \dots & x_p^{(n)} \end{bmatrix}$$

Hva er egentlig fordelene, utover den matematiske elegansen?
En viktig grunn er at matriseoperasjoner kan utføres *mye raskere* enn sekvensiell behandling av datapunkter, takket være dedikerte optimaliseringer i moderne CPU- og GPU-arkitekturer.

Og at lineær regresjon kunne da utføres i ett steg:

$$\mathbf{y} = \mathbf{X} \mathbf{w} + \mathbf{b}$$

Matriseprodukt

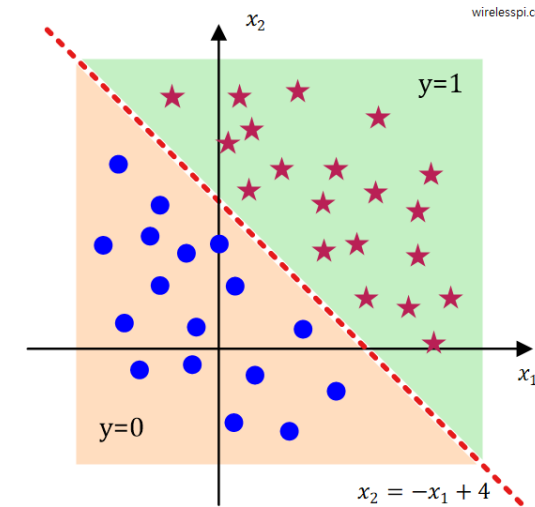
Vi kan sjekke at formelen $\mathbf{y} = \mathbf{X} \mathbf{w} + \mathbf{b}$ faktisk gir oss riktig resultat:

$$\mathbf{y} = \mathbf{X} \mathbf{w} + \mathbf{b} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_p^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_p^{(2)} \\ \dots & \dots & \dots & \dots \\ x_1^{(n)} & x_2^{(n)} & \dots & x_p^{(n)} \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_p \end{bmatrix} + \mathbf{b}$$

Som forventet får vi her en vektor $\mathbf{y}$ av lengde n (antall datapunkter), hvor hver dimensjon i inneholder resultatet av en lineær regresjon med datapunkt $\mathbf{x}_i$, vektene $\mathbf{w}$ og konstantleddet $\mathbf{b}$

$$= \begin{bmatrix} w_1 x_1^{(1)} + w_2 x_2^{(1)} + \dots + w_p x_p^{(1)} \\ w_1 x_1^{(2)} + w_2 x_2^{(2)} + \dots + w_p x_p^{(2)} \\ \vdots \\ w_1 x_1^{(n)} + w_2 x_2^{(n)} + \dots + w_p x_p^{(n)} \end{bmatrix} + \mathbf{b} = \begin{bmatrix} \sum_{i=1}^p w_i x_i^{(1)} + b \\ \sum_{i=1}^p w_i x_i^{(2)} + b \\ \vdots \\ \sum_{i=1}^p w_i x_i^{(n)} + b \end{bmatrix}$$

I dag skal vi snakke om 2 temaer:



Lynkurs i matriseregning

Vektorer og matriser brukes
overalt i maskinlæring – *det
lønner seg å kunne
grunnleggende matriseregning!*

Logistisk regresjon

- En utvidelse av lineær regresjon som kan brukes til *klassifisering* i stedet for regresjon
- En viktig «brikke» i mange ML-modeller

Plan

- Modellen
- Utvidelse til > 2 klasser
- Læring
- Prediksjon
- Oppsummering

Plan

- **Modellen**

- Utvidelse til > 2 klasser
- Læring
- Prediksjon
- Oppsummering

Logistisk regresjon

Logistisk regresjon er en *probabilistisk* klassifikasjonsmodell: den gir oss en sannsynlighet

$$P(c | \mathbf{x})$$

for at et datapunkt $\mathbf{x}$ tilhører klassen c

Den vertikalen linjen brukes til å uttrykke en *betinget sannsynlighet*: vi beregner sannsynligheten for en klasse c *gitt* at vi observerer inputtrekkene $\mathbf{x}$

Som for lineær regresjon er datapunktet $\mathbf{x}$ representert gjennom ulike *inputtrekk*

Valgt ut fra ett sett $\mathcal{C}$ av mulige klasser definert på forhånd

Merk: Klassene antas å være gjensidig utelukkende: hvis $\mathbf{x}$ tilhører klassen c kan punktet ikke samtidig tilhøre andre klasser $c' \neq c$!

Eksempler

Mulige outputklasser



Gitt en *epost* representert gjennom ulike trekk (antall ord, forekomst av ord som “Viagra”, “prince”, osv.), hva er sannsynligheten for at eposten er *spam*?

$$\mathcal{C} = \left\{ \begin{array}{l} \text{spam,} \\ \text{ikke spam} \end{array} \right\}$$



Gitt ulike opplysninger om en *person* (alder, kjønn, kosthold, osv.), hva er sannsynligheten for at vedkommende utvikler *kreft* i de neste 5 årene?

$$\mathcal{C} = \left\{ \begin{array}{l} \text{kreft,} \\ \text{ikke kreft} \end{array} \right\}$$



Gitt ulike opplysninger om en *student* (forkunnskap, deltakelse i seminarer og labber, innsats, osv.), hva blir deres sannsynlige *karakter* for IN1160 ?

$$\mathcal{C} = \left\{ \begin{array}{l} A, \\ B, \\ C, \\ D, \\ E, \\ F \end{array} \right\}$$

Binær logistisk regresjon

- Vi starter med det enkleste tilfellet, kalt *binær logistisk regresjon*, hvor vi må velge mellom **to** mulige klasser c_1 og c_2
 - For eksempel eposter klassifisert som {spam, ikke spam}
- Da trenger modellen bare å predikere én sannsynlighet.
- Hvorfor det? Når $|C| = 2$ har vi to valg som utelukker hverandre: c_1 eller c_2 . Derfor har vi per definisjon $P(c_2 | \mathbf{x}) = 1 - P(c_1 | \mathbf{x})$!

Dette kommer fra selve definisjonen av en sannsynlighetsfordeling:
sannsynlighetene for alle mulige verdier C må være mellom 0 og 1 og summeres opp til 1.

Binær logistisk regresjon

- Forrige uke snakket vi om lineær regresjon, som kunne brukes til å predikere et tall y basert på ulike trekk $\mathbf{x}$:

$$y = \sum_{i=1}^p w_i x_i + b$$

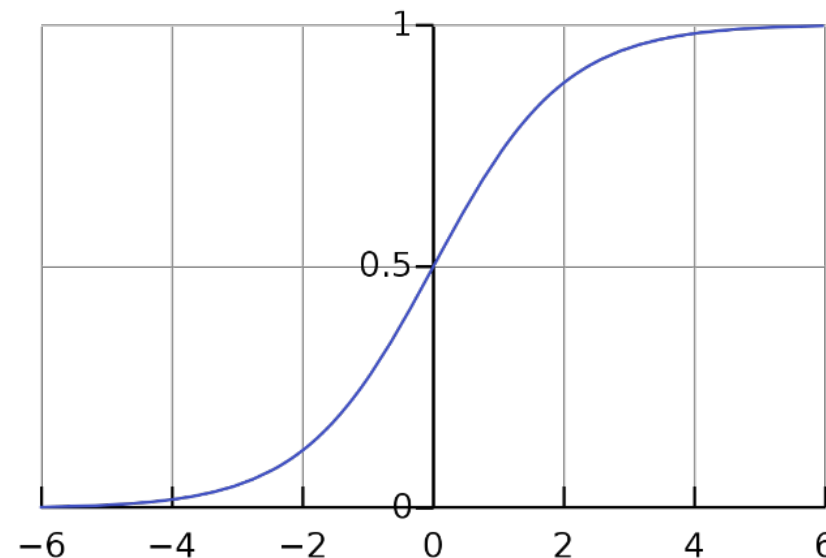
- Kunne vi kanskje også bruke lineær regresjon til å gi oss sannsynligheten $P(c_1 | \mathbf{x})$?
- **Utfordring:** en lineær regresjon gir oss en output y som ligger i $[-\infty, +\infty]$, men vi må ha et tall i $[0, 1]$!

Binær logistisk regresjon

- Heldigvis kan utfordringen løses med en liten endring i formelen: vi beholder $\sum_{i=1}^p \mathbf{w}_i x_i + b$, men etter det *tar vi resultatet gjennom en funksjon σ*
- Funksjonen σ kalles en *sigmoid funksjon* og defineres som:

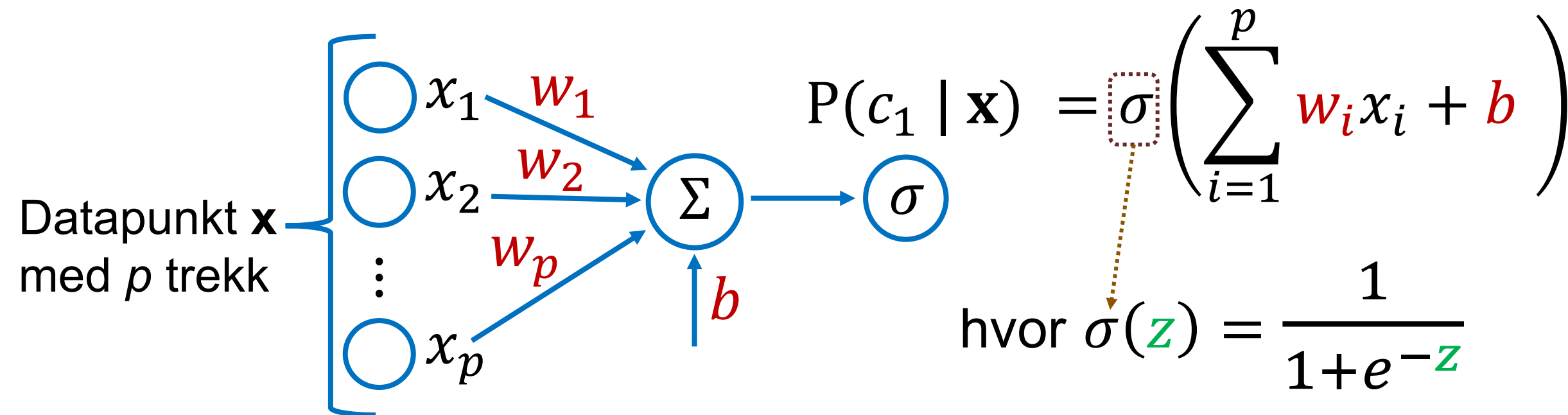
Merk: her bruker vi z som variabelnavn i funksjonen for å unngå forvirring med $\mathbf{x}$, som vi har brukt for inputtrekkene

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



- Rollen til σ er å «komprimere» y til en verdi i $[0, 1]$

Binær logistisk regresjon



Som i lineær regresjon har vi vektene $\mathbf{w}$ og konstantleddet b som parametere. Men logistisk regresjon tar resultatet av den vektete summen + konstantledd *gjennom en sigmoid-funksjon*

Binær logistisk regresjon

- Sigmoid-funksjonen σ “tvinger” regresjonsresultatet til å være en sannsynlighetsverdi $\in [0,1]$:

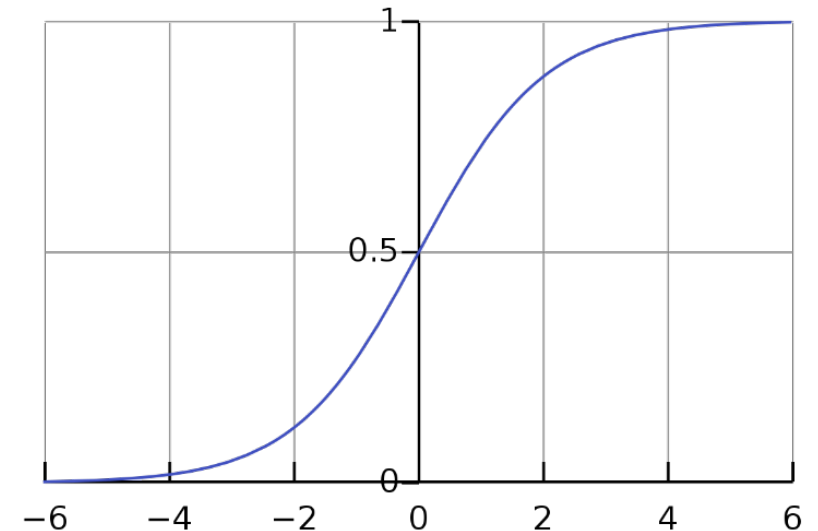
$$P(c_1 | \mathbf{x}) = \sigma\left(\sum_{i=1}^p \mathbf{w}_i x_i + b\right) \quad \text{hvor } \sigma(\mathbf{z}) = \frac{1}{1+e^{-\mathbf{z}}}$$

- Hvis $\sum_{i=1}^p \mathbf{w}_i x_i + b$ er
 - Veldig lavt (e.g. -100)
 - 0
 - Veldig høyt (e.g. 100)

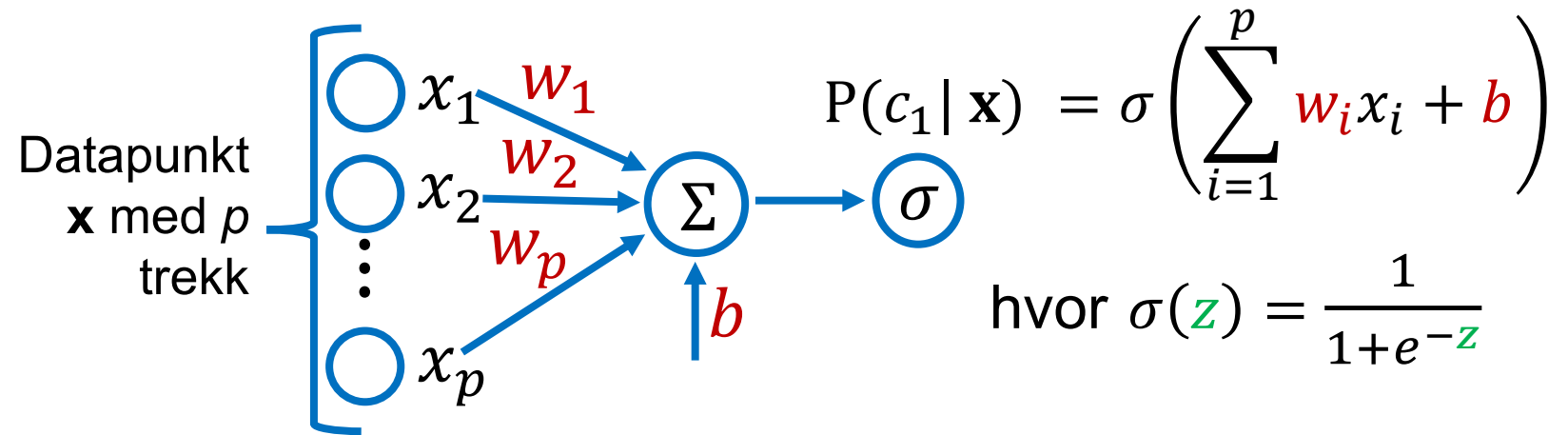
→ $P(c_1 | \mathbf{x}) \approx 0$

→ $P(c_1 | \mathbf{x}) = 0.5$

→ $P(c_1 | \mathbf{x}) \approx 1$



Mentimeter



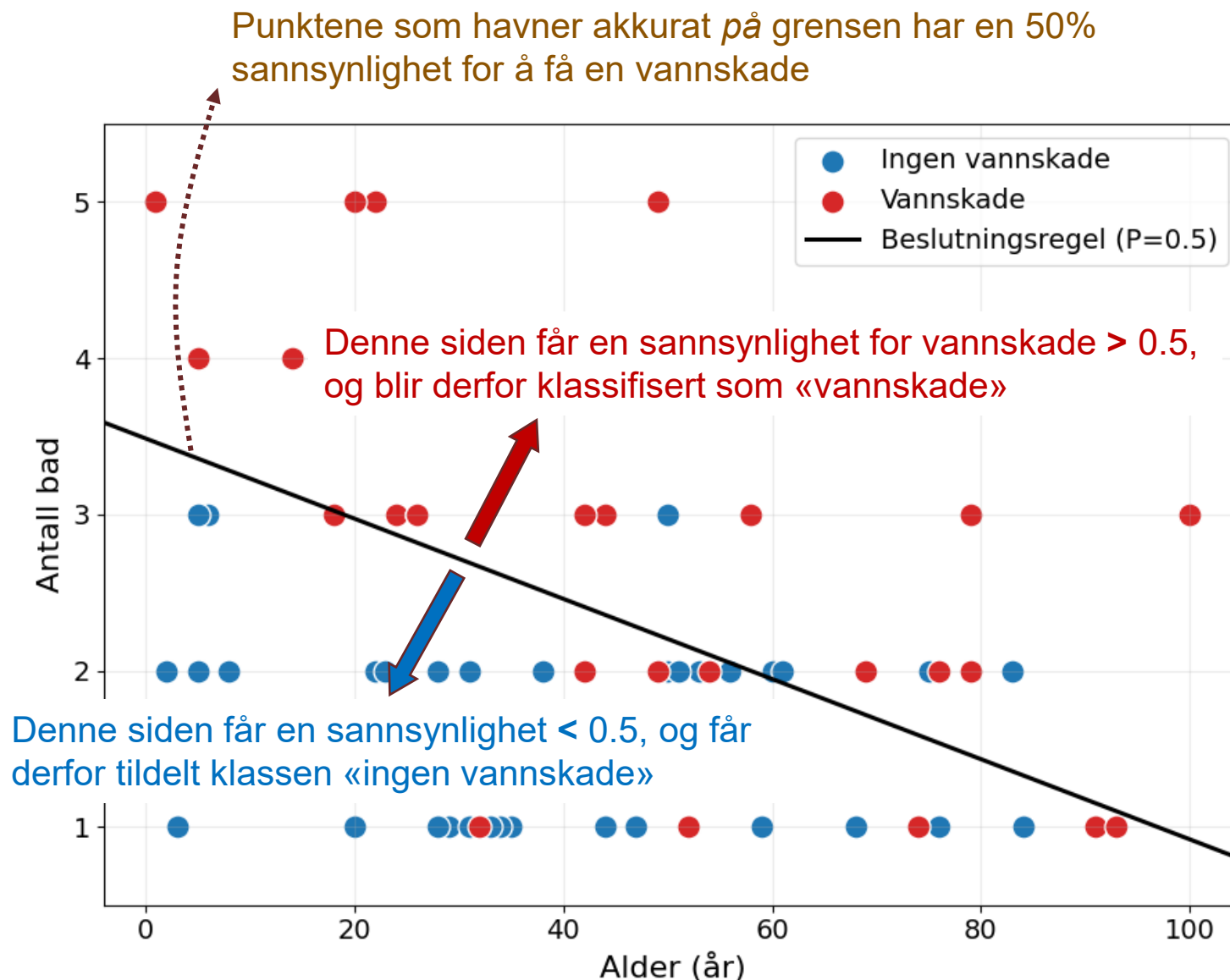
- Vi ønsker å forutsi om en bolig vil få en vannskade de neste 10 årene, basert på *antall bad* og *alder* (antall år siden boligen ble bygd).
- Vi har estimert en logistisk regresjonsmodell med følgende parametere:

$$w_{bad} = 0.6 \quad w_{alder} = 0.02 \quad b = -2$$

- **Hva er sannsynligheten for at en bolig med 2 bad fra 1956 vil få en vannskade de neste ti år?**

Beslutningsgrense

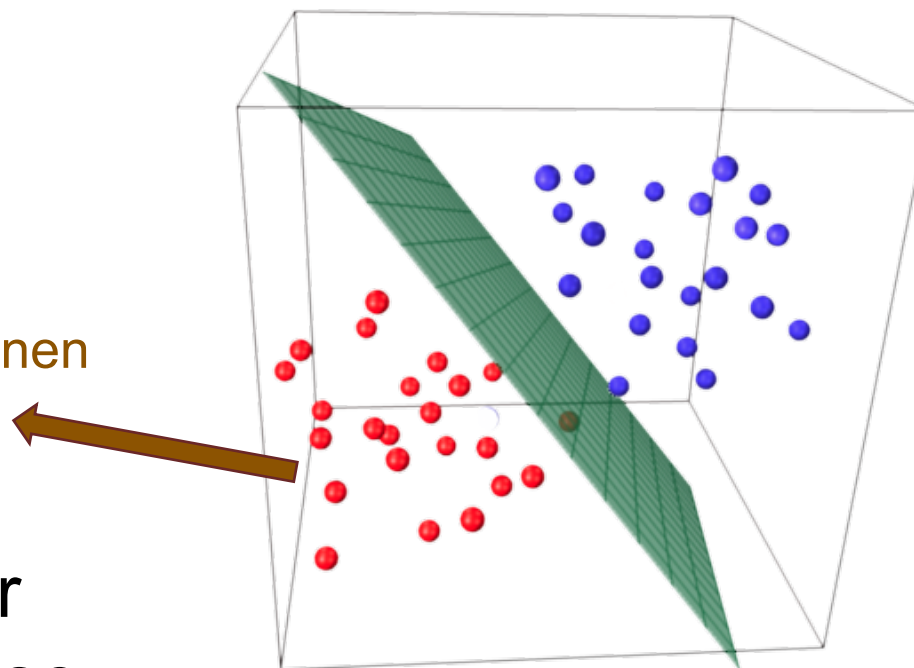
- Vi kan tegne en såkalt **beslutningsgrense** (eng. *decision boundary*) for å se hva modellen gjør
- Beslutningsgrensen viser hvordan modellen splitter inputtrekkrommet i to:
 - en del hvor $P(c_1|\mathbf{x}) > P(c_2|\mathbf{x})$
 - Og en del hvor $P(c_1|\mathbf{x}) < P(c_2|\mathbf{x})$



Beslutningsgrense

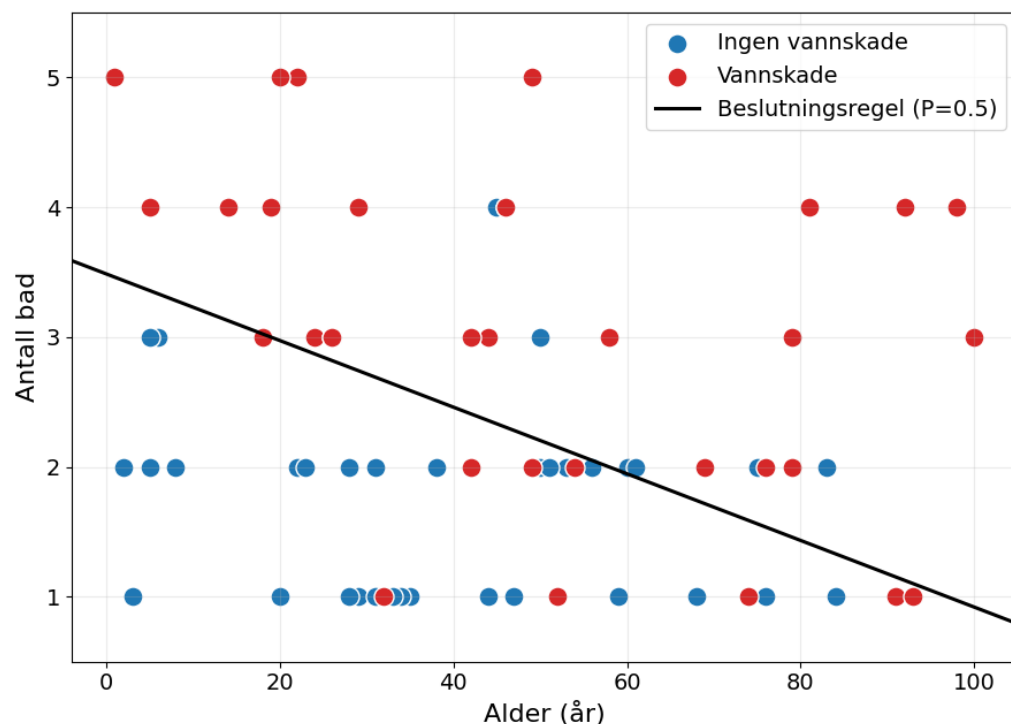
- Med $n > 2$ trekk er beslutningsgrensen en *hyperplan* i et n -dimensjonalt rom
- Sannsynligheten for å tilhøre enten c_1 eller c_2 øker med *avstanden* til beslutningsgrense
 - Eller med andre ord: punktene som ligger nær grensen er de som er mest “usikre”

Punkter på denne siden av planen vil være klassifisert som **rødt**



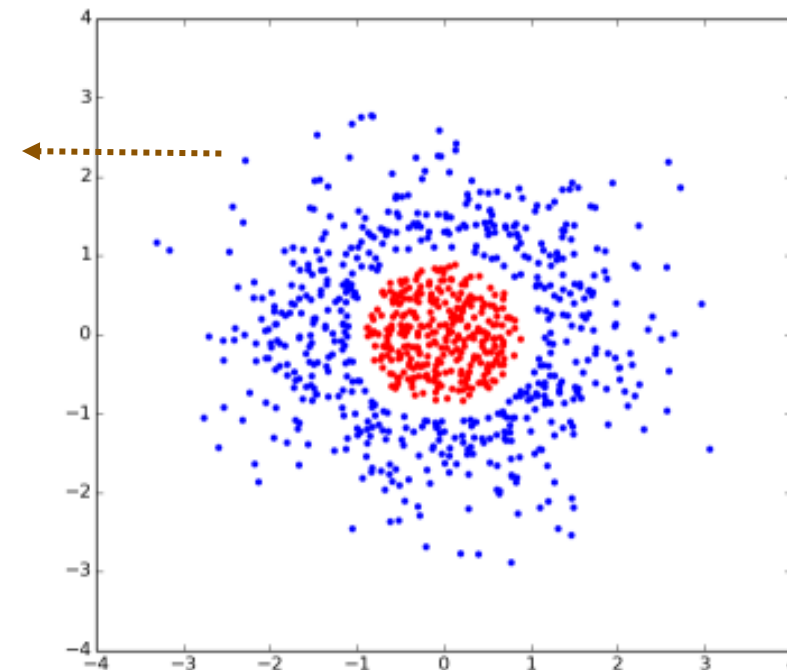
Linearitet

Logistisk regresjon er en *lineær* modell (i likhet med lineær regresjon):



Logistisk regresjon egner seg for mange klassifiseringsproblemer, men ikke alle:

Her har vi et datasett som ikke er *lineært separerbar*: Det finnes mao. ingen rett linje som kan skille godt mellom de to klassene (uten å endre på inputtrekkene)



Plan

- Modellen
- **Utvidelse til > 2 klasser**
- Læring
- Prediksjon
- Oppsummering

Logistisk regresjon med > 2 klasser

- Når outputklassene $\mathcal{C} = \{c_1, c_2, \dots, c_C\}$ består av flere enn 2 klasser kan vi ikke lenger bruke en sigmoid
- I stedet regner vi ut først en vektet sum *per klasse* c_j :

$$z_j = \sum_{i=1}^p w_{ij} x_i + b_j$$

Merk: vi har nå egne parameter w_j og konstantledd b_j for hver klasse c_j

- Og normaliserer deretter z -verdiene gjennom en såkalte softmax :

$$P(c_j | \mathbf{x}) = \text{softmax}(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^C e^{z_k}}$$

Softmax brukes til å sikre at resultatet blir en riktig sannsynlighetsfordeling

Softmax-eksempel

- softmax-funksjonen ser litt skummelt ut - men er egentlig ikke så vanskelig!
- Vi kan ta et eksempel med 3 klasser. La oss først anta de vektete summene $\sum_{i=1}^p w_{ij}x_i + b_j$ har gitt oss de tre tallene

$$\mathbf{z} = [2.3 \quad 0.4 \quad -1.4] \longrightarrow \text{Merk: Disse } \mathbf{z}\text{-verdiene kalles ofte "logits" i maskinl ring. De uttrykker en slags score som m  deretter normaliseres til en sannsynlighet}$$

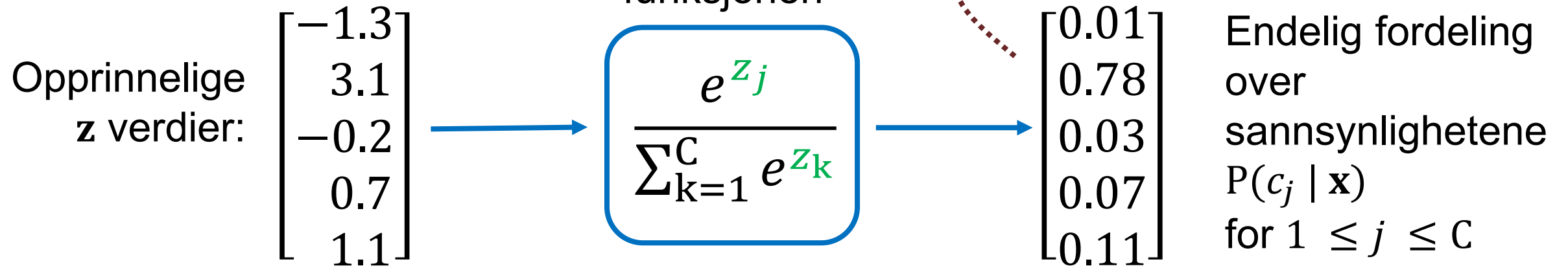
- Vi starter med   regne $e^{\mathbf{z}}$ for de tre verdiene i $\mathbf{z}$:

$$[9.97 \quad 1.49 \quad 0.25]$$

- Vi summerer opp disse tre tallene: $\sum_{k=1}^C e^{\mathbf{z}_k} = 11.71$
- Og vi bruker resultatet til   normalisere de tre $e^{\mathbf{z}}$ tallene:

$$[0.85 \quad 0.13 \quad 0.02]$$

Softmax



- softmax har samme rolle som sigmoid funksjon, bare med flere enn 2 klasser: den brukes til å “komprimere” resultatene fra de vektete summene til en sannsynlighetsfordeling
- softmax er en viktig funksjon i maskinlæring, og brukes i mange andre modeller enn logistisk regresjon!

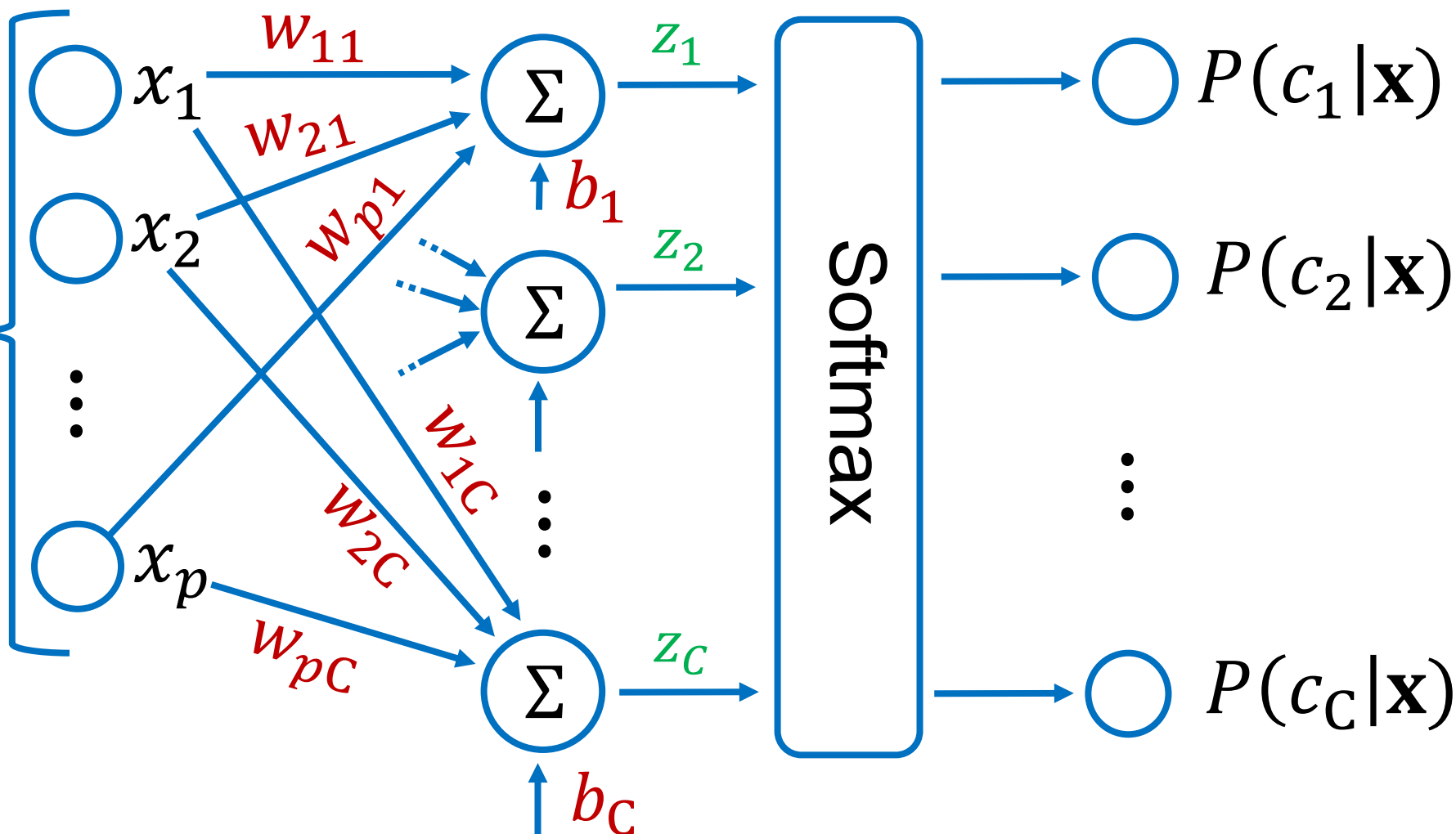
Multinomisk logistisk regresjon

Sannsynlighetsfordeling
over klassene $c_1, c_2, \dots, c_C$:

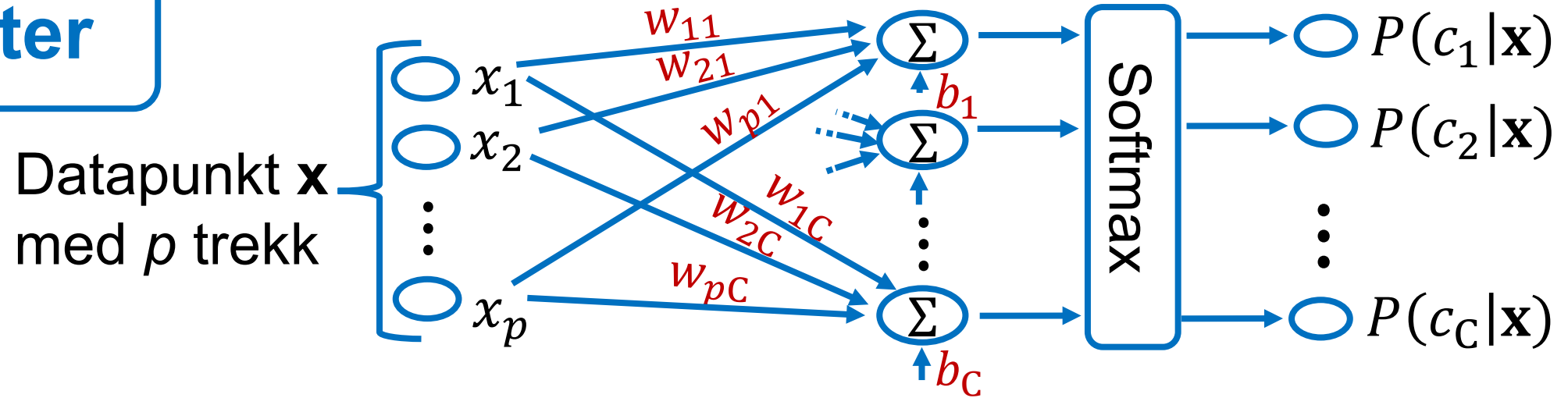
«Multinomisk
logistisk regresjon»
er navnet på
logistisk regresjon
med > 2 klasser

Et annet navn er
«softmax» regresjon

Datapunkt $\mathbf{x}$
med p trekk



Mentimeter

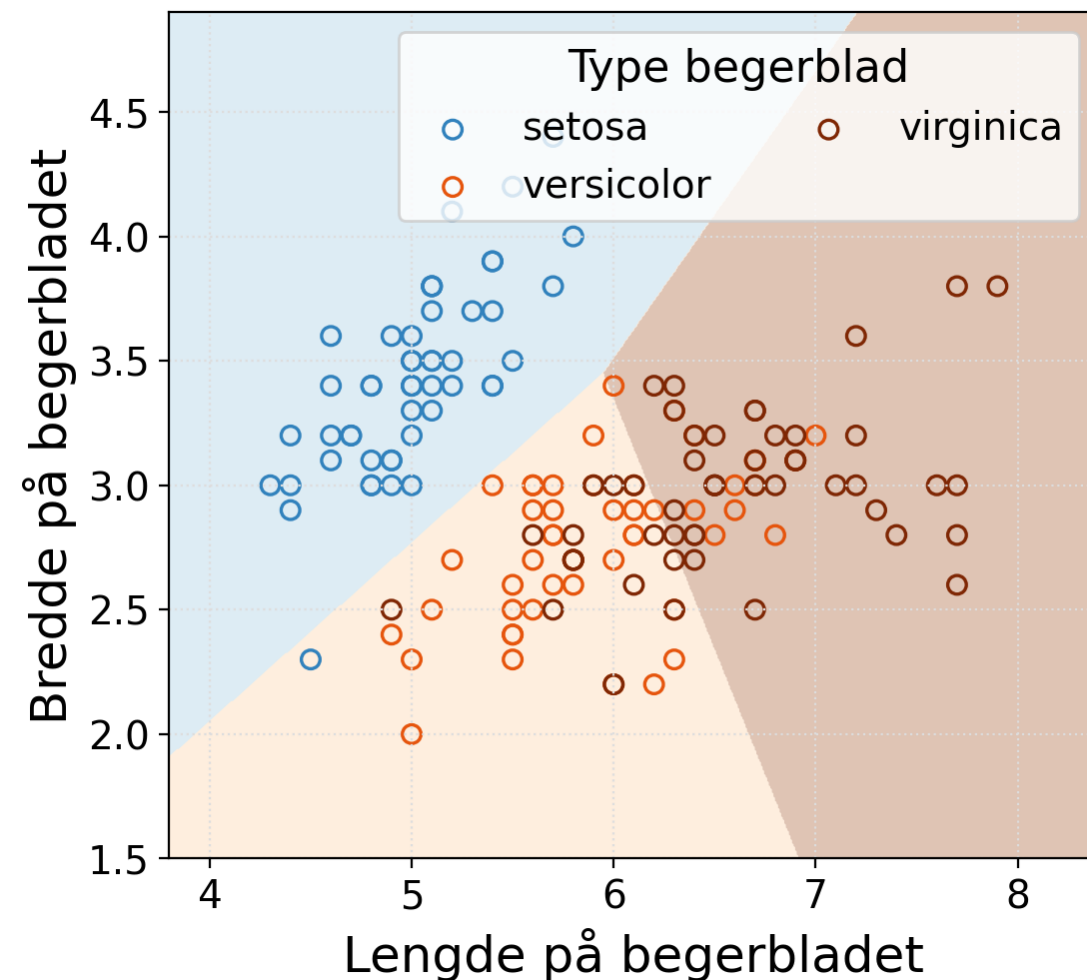


- I multinomisk logistisk regresjon har vi en vektvektor $\mathbf{w}_j$ og konstantleddet b_j for hver klasse j med $1 \leq j \leq C$
- *La oss si at vi har et klassifiseringsoppgave med 4 klasser, 10 inputtrekk per datapunkt, og 10 000 eksempler i treningssettet.*

→ **Hvor mange parametere har vi til sammen?**

Beslutningsgrenser

- Multinomisk logistisk regresjon deler modellen inputtrekkrommet i “regioner” hvor en klasse er mer sannsynlig enn de andre
- Grensene består av *rette linjer* (modellen er fortsatt lineær)
- Punktene som befinner seg lengre fra klassens regionsgrenser er “sikrere” (=høyere sannsynlighet)

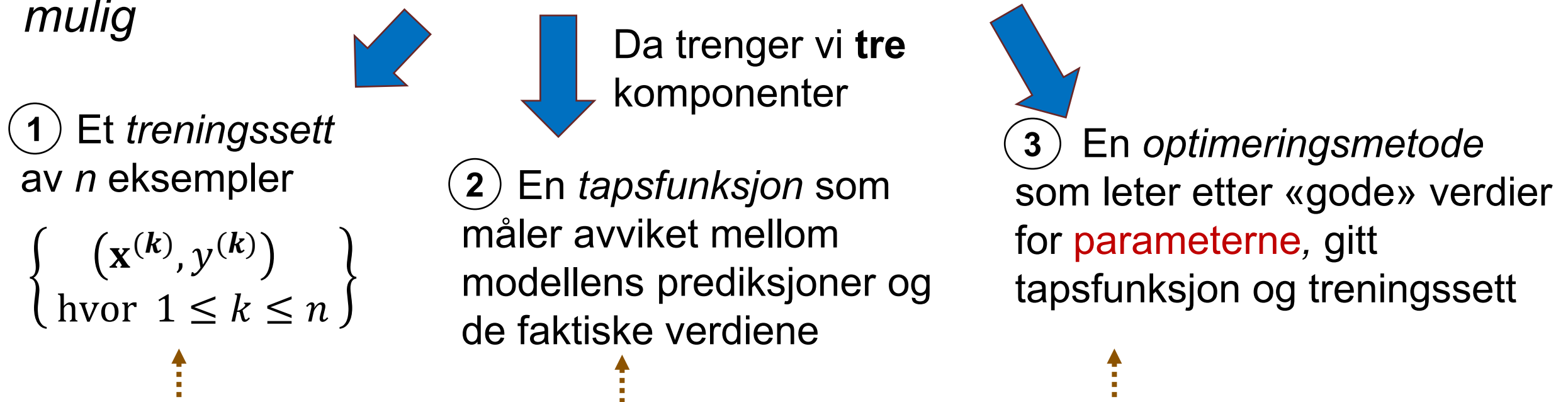


Plan

- Modellen
- Utvidelse til > 2 klasser
- **Læring**
- Prediksjon
- Oppsummering

Læring (fra forrige uke)

Intuisjon: finne verdiene for **parameterne** som gjør at “feilene” mellom hva modellen forutsier og fasitverdiene i treningssettet blir så *små* som *mulig*



Det eneste som endrer seg fra prosedyren vi så forrige uke er ② **tapsfunksjonen**, som er forskjellig for lineær og logistisk regresjon. ① og ③ er omtrent de samme.

Tapsfunksjonen

- La $(\mathbf{x}^{(k)}, y^{(k)})$ betegne et treningseksempel med inputtrekk $\mathbf{x}^{(k)}$ og den «ekte» klassen $y^{(k)}$ for eksempelet
- Med kun to klasser c_1 og c_2 bruker man typisk **binær kryssentropi** (*binary cross-entropy* eller BCE) som tapsfunksjon:

$$l_{\text{BCE}}(\hat{y}^{(k)}, y^{(k)}) = \begin{cases} -\log(\hat{y}^{(k)}) & \text{hvis } y^{(k)} = c_1 \\ -\log(1 - \hat{y}^{(k)}) & \text{hvis } y^{(k)} = c_2 \end{cases}$$

$\hat{y}^{(k)} = P(c_1 | \mathbf{x}^{(k)})$ betegner sannsynligheten for klassen c_1 predikert av modellen basert på trekkene $\mathbf{x}^{(k)}$

Eksempel: Hvis prediksjonen fra modellen er $\hat{y} = 0.8$ og $y = c_1$, blir tapet $-\log 0.8 = 0.22$.
Om $y = c_2$ er tapet mye større: $-\log 0.2 = 1.61$!

Tapsfunksjonen

En utvidelse av BCE for C klasser (med $C > 2$) er **kategorisk kryssentropi** (*categorical cross-entropy* eller CCE), definert som:

$$l_{\text{CCE}}(\hat{\mathbf{y}}^{(k)}, \mathbf{y}^{(k)}) = - \sum_{i=1}^C y_i^{(k)} \log(\hat{y}_i^{(k)})$$

↙
Sannsynlighetsfordelingen
 $[\hat{y}_1^{(k)}, \dots, \hat{y}_C^{(k)}]$ over klassene
predikert av modellen basert
på trekkene $\mathbf{x}^{(k)}$

Husk at «hatten» viser at den
er en prediksjon, ikke fasiten!

↓
Binær koding av fasitverdien:

$$y_i^{(k)} = \begin{cases} 1 & \text{hvis } y^{(k)} = i \\ 0 & \text{ellers} \end{cases}$$

↘
 $\hat{y}_i^{(k)} = P(c_i | \mathbf{x}^{(k)})$ er
sannsynligheten for klassen
 c_i predikert av modellen ut
fra inputtrekkene $\mathbf{x}^{(k)}$

Tapsfunksjonen

- Vi kan deretter beregne det gjennomsnittlige tapet på hele treningssettet (n eksempler):

$$\hat{L} = \frac{1}{n} \sum_{k=1}^n l(\hat{y}^{(k)}, y^{(k)})$$

- Som for lineær regresjon kan vi da bruke *stokastisk gradientnedstigning* til å lete etter verdiene for parameterne $\mathbf{w}_1, b_1, \dots, \mathbf{w}_C, b_C$ som minimiserer dette tapet $\hat{L}$

Husk: Med “binær” logistisk regresjon har modellen kun én vektvektor og ett konstantledd, mens i multinomisk logistisk regresjon (med > 2 klasser) har vi én vektvektor og ett konstantledd *per klasse*

$$\text{Husk: } l(\hat{\mathbf{y}}^{(k)}, \mathbf{y}^{(k)}) = - \sum_{i=1}^C y_i^{(k)} \log(\hat{y}_i^{(k)})$$

- Vi ønsker å klassifisere boliger i tre klasser: *leilighet*, *tomannsbolig* og *enebolig*
- Vi har samlet et lite treningssett med 2 datapunkter:
$$\left\{ \begin{array}{l} (x^{(1)} = [0 \quad 1.2 \quad 4.5], \quad y^{(1)} = \text{leilighet}), \\ (x^{(2)} = [1 \quad 0.3 \quad -2], \quad y^{(2)} = \text{tomannsbolig}) \end{array} \right\}$$
- Basert på dette treningssettet har vi lært en multinomisk logistisk regresjon, som gir oss de følgende prediksjonene for de 2 datapunktene:
$$\begin{aligned} \hat{\mathbf{y}}^{(1)} &= [0.87 \quad 0.09 \quad 0.04] \\ \hat{\mathbf{y}}^{(2)} &= [0.44 \quad 0.35 \quad 0.21] \end{aligned}$$

Sannsynlighetene for henholdsvis “*leilighet*”, “*tomannsbolig*” og “*enebolig*”
- **Hva er det gjennomsnittlige tapet på treningssettet?**

Plan

- Modellen
- Utvidelse til > 2 klasser
- Læring
- **Prediksjon**
- Oppsummering

Prediksjon

- Logistisk regresjon er en *probabilistisk* klassifiseringsmodell: som output får vi en *sannsynlighetsfordeling* over alle mulige klassene
- Typisk vil vi velge den mest sannsynlige klassen, altså

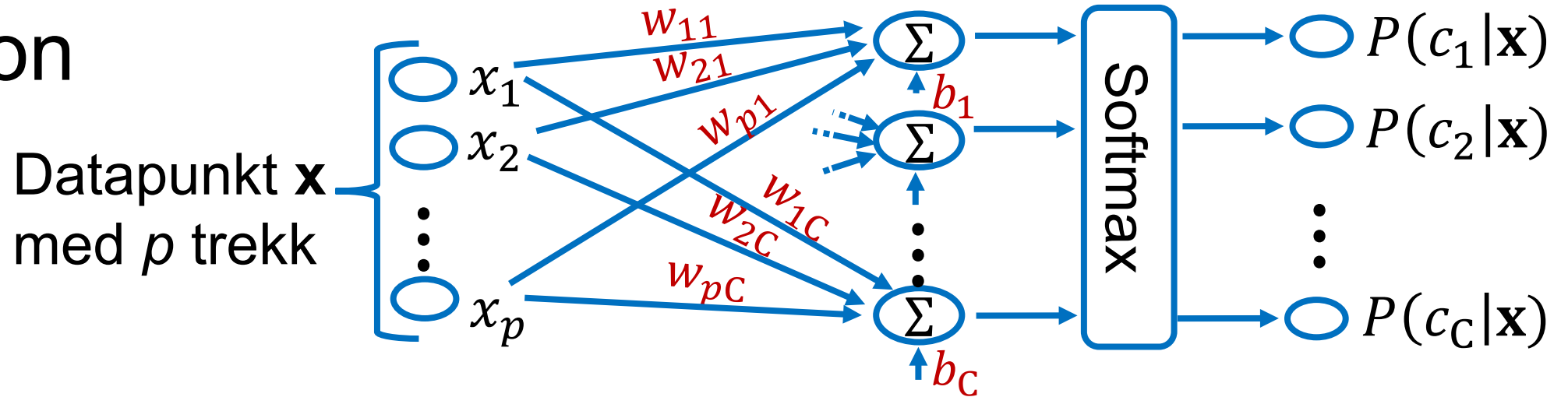
$$c^* = \operatorname{argmax}_{c \in \mathcal{C}} P(c | \mathbf{x})$$

I maskinlæring bruker vi ofte tegnet “*” til å betegne “den beste verdien”

“argmax” angir at vi skal finne argumentet (her $c \in \mathcal{C}$) som gjør at uttrykket i argmax (her $P(c | \mathbf{x})$) blir så høyt som mulig.

- Men andre strategier er mulige.
Eksempel: i anvendelser hvor feile beslutninger har en høy kostnad kan det lønne seg å avstå helt fra å klassifisere hvis $P(c | \mathbf{x})$ kommer under en viss terskel

Prediksjon



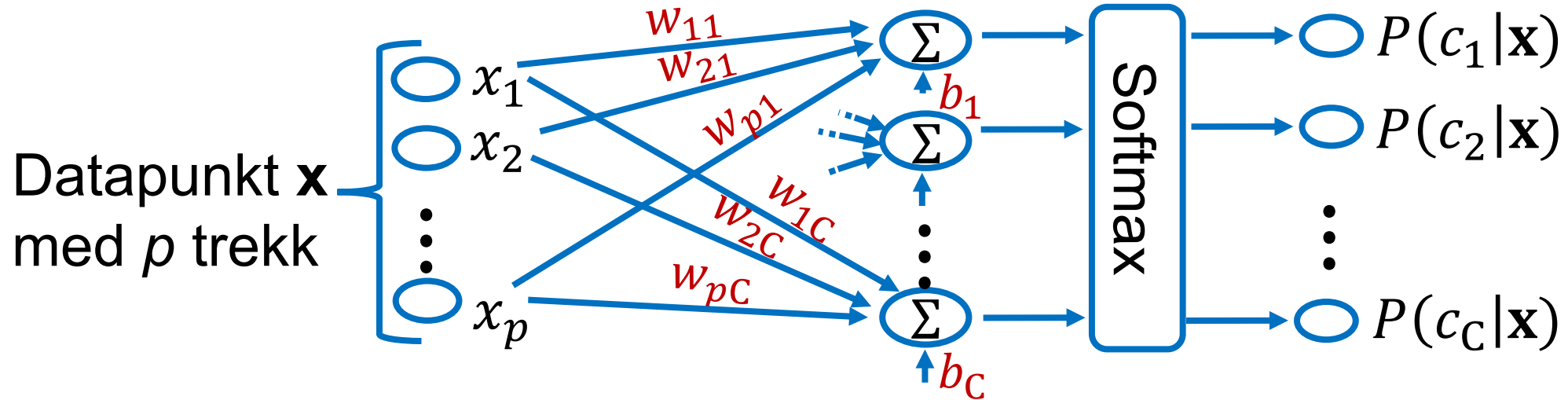
En fordel av logistisk regresjon er at vi kan forstå hvordan ulike trekk og tilsvarende vektorer bidrar til sluttprediksjonen:

- Hvis w_{ij} er > 0 : Hvis x_i øker $\rightarrow P(c_j | \mathbf{x})$ øker også
- Hvis w_{ij} er < 0 : Hvis x_i øker $\rightarrow P(c_j | \mathbf{x})$ minsker
- Hvis $w_{ij} \approx 0$: $P(c_j | \mathbf{x})$ blir (omtrent) ikke påvirket av x_i
- Og *større* (positive eller negative) vektorer $\rightarrow$ *større* påvirkning

Plan

- Modellen
- Utvidelse til > 2 klasser
- Læring
- Prediksjon
- **Oppsummering**

Oppsummering



- Logistisk regresjon er en *lineær, probabilistisk* modell for å klassifisere datapunkter i ulike klasser
- To varianter:
 - Binær logistisk regresjon når antall klasser = 2 → bruk av sigmoid
 - Multinomisk logistisk regresjon ellers → bruk av softmax